

实验课五：智能体个体模拟



本次实验不允许大家使用 AI 辅助编程（包括但不限于直接聊天、Copilot、Vibe Coding）。

最终提交结果会进行脚本核验，如发现出现作弊现象（包括但不限于【使用非指定的大模型进行数据构造/训练/推理】，【擅自修改/调整训练数据】），以零分处理。

1. 实验概述

本实验围绕基于大语言模型的个体模拟展开，目标是让大家完整经历一次从角色记忆构建、Memory-Augmented Agent、监督微调 / 偏好优化数据构造、模型训练、综合评测分析到结果汇报的实验流程。

基础模型统一使用 Qwen2.5-1.5B-Instruct，记忆检索模型采用 Qwen3-Embedding-0.6B。

本次实验分为两个层次：

- 主线任务：给定一个具有稳定 Persona 的目标个体，使模型在访谈场景中能够产生与该个体一致的行为表现。这里的一致性包括事实信息是否正确，说话风格、价值观倾向是否对齐。
- 附加任务：在 Task 4 中加入通用指令能力评测集，用于分析模型在适配目标角色后是否仍能维持通用指令执行能力。

2. 论文阅读与分享

需要阅读与个体模拟相关的代表性工作，课堂分享内容建议包括：

- 论文试图解决什么问题；
- 核心方法是什么；
- 关键实验结论是什么；

编号	主题	论文列表
Group 1	角色扮演	In-Context Impersonation Reveals Large Language Models' Strengths and Biases
		Character-LLM: A Trainable Agent for Role-Playing

Group 2	社交行为模拟	SOTOPIA: Interactive Evaluation for Social Intelligence in Language Agents
		AgentSense: Benchmarking Social Intelligence of Language Agents through Interactive Scenarios
Group 3	长期个体模拟	Generative Agents: Interactive Simulacra of Human Behavior
		Humanoid Agents: Platform for Simulating Human-like Generative Agents
Group 4	社会学/心理学实验	Using Large Language Models to Simulate Multiple Humans and Replicate Human Subject Studies
		Can Large Language Model Agents Simulate Human Trust Behavior?

3. 实验数据与评估说明

本次实验会提前发布已经整理好的数据文件，大家基于发布的数据文件进行后续任务。

3.1 角色个体模拟数据

本实验使用一个合成的角色个体模拟数据集。数据集中包含 68 个可定制角色、1,360 个真实场景和 13,971 段多轮角色扮演对话。角色由 `career`、`aspiration`、`traits`、`skills` 等预定义属性扩展出个人信息、社会关系和历史互动。角色个体模拟数据文件结构如下：

代码块

```
1  individual_simulation_data/
2  |— characters/
3  |   |— wiki_{character_name}.txt
4  |   └─ ...
5  |— scene/
6  |   |— generated_agent_scene_{character_name}.json
7  |   └─ ...
8  |— dialogue/
9  |   |— generated_agent_dialogue_{character_name}.json
10 |   └─ ...
11 └─ interview/
12     |— generated_agent_interview_{character_name}.json
13     └─ ...
```

其中：

- `characters/`：每个文件对应一个角色的 Persona 文本，包含角色身份、职业、性格、偏好、社会关系和生活背景等信息；
- `scene/`：每个文件包含该角色相关的场景设定，例如地点、状态、情绪和对话主题；
- `dialogue/`：每个文件包含该角色参与的多轮角色扮演对话，可用于构建 Memory Pool、提取典型表达或构造训练样本；
- `interview/`：每个文件包含该角色的采访问题，是本实验角色采访问答任务的**主要评测输入**。评估时请采用interview中的问题，对齐到对应的角色上。

3.2 通用指令能力评测数据

本实验还会发布一个通用指令能力评测文件，用于 Task 4 中分析角色适配对通用指令跟随能力的影响。该文件为 JSONL 格式，每行是一条评测样本，只保留必要字段：

代码块

```
1  {"key": "sample_0001", "prompt": "..."} 
```

其中：

- `key`：样本唯一标识；
- `prompt`：需要模型完成的自然语言指令。

模型需要根据 `prompt` 生成回答，并在提交预测文件时保留样本标识，便于评测脚本对齐输入与输出。

3.3 评估说明

由于本次角色个体模拟任务属于开放式问答评估，标准答案并不唯一，因此角色模拟部分采用 LLM-as-a-Judge 形式进行评分。为防止针对评估 Prompt 或规则进行 hack，实验说明中只提供简略版本的评估维度描述。大家在自行实验和报告分析时，可以参考这些维度自行编写评估用 Prompt，并需要在报告中详细说明所使用的 Judge 模型与评分 Prompt。

角色模拟评估主要包含以下四个维度：

- Memorisation：回答是否正确使用角色背景、经历、关系和历史记忆。
- Values：回答是否符合角色的价值观、偏好和行为倾向。
- Personality：回答是否保持角色的性格特征、语气和说话风格。
- Hallucination：回答是否避免与角色设定或已知记忆相矛盾的内容。

4. 任务说明

4.1 Task 0: Memory 核心代码实现

Task 0 是后续任务的基础。本任务要求大家实现一个轻量级角色记忆检索模块。该模块接收当前用户问题和某个角色的 Memory Pool，返回最相关的若干条记忆，用于 Agent 回答或训练数据构造。请补全下列代码文件中的空缺部分，完成 10-20 行左右的核心检索代码。补全后，该文件应当能够直接运行，并打印一组可检查的检索结果。若实现正确，程序应输出检索结果和检查项。检查项中应全部显示 `PASS`。

代码块

```
1 Retrieved memories:
2 1. m_space ...
3 2. m_award ...
4
5 Checks:
6 [PASS] top-1 memory is m_space
7 [PASS] top-2 memory is m_award
```

代码块

```
1 import numpy as np
2 import re
3
4 def normalize_text(text):
5     text = text.lower()
6     text = re.sub(r"^[a-zA-Z0-9\u4e00-\u9fa5\s]", " ", text)
7     return text
8
9 def lexical_overlap(query, memory_text):
10    q_words = set(normalize_text(query).split())
11    m_words = set(normalize_text(memory_text).split())
12    if len(q_words) == 0:
13        return 0.0
14    return len(q_words & m_words) / len(q_words)
15
16 def retrieve_memories(
17     query,
18     query_embedding,
19     memories,
20     top_k=5,
21     alpha=0.65,
22     beta=0.25,
23     gamma=0.10
24 ):
25     """
26     memories: List[Dict]
27     每条 memory 至少包含:
28     {
```

```

29         "memory_id": str,
30         "text": str,
31         "embedding": List[float],
32         "importance": float,
33         "type": str
34     }
35
36     score = alpha * semantic_similarity
37           + beta * lexical_overlap
38           + gamma * importance
39     """
40
41     # ===== 代码开始 =====
42     # TODO: 计算每条 memory 的 semantic score、lexical score、importance score
43     # TODO: 按综合 score 排序
44     # TODO: 返回 top_k 条 memory, 并在返回结果中保留 score
45     # 提示:
46     # 1. semantic_similarity 使用 cosine similarity
47     # 2. lexical score 使用 lexical_overlap(query, memory["text"])
48     # 3. importance score 使用 memory["importance"]
49     # 4. 返回结果中每条 memory 应包含 score / semantic_score / lexical_score /
importance_score
50     # ===== 代码结束 =====
51
52     return selected_memories
53
54 def print_check(name, passed):
55     status = "PASS" if passed else "FAIL"
56     print(f"[{status}] {name}")
57
58 def run_demo():
59     query = "How was your first Mars mission?"
60     query_embedding = np.array([1.0, 0.0])
61
62     memories = [
63         {
64             "memory_id": "m_space",
65             "type": "experience",
66             "text": "I still remember my first Mars mission and the zero
gravity training before launch.",
67             "embedding": [0.99, 0.10],
68             "importance": 0.7,
69         },
70         {
71             "memory_id": "m_food",
72             "type": "hobby",
73             "text": "I enjoy cooking pasta for my friends during holidays.",

```

```

74         "embedding": [0.0, 1.0],
75         "importance": 0.9,
76     },
77     {
78         "memory_id": "m_award",
79         "type": "career",
80         "text": "The NASA award ceremony after the mission made me proud.",
81         "embedding": [0.70, 0.70],
82         "importance": 0.8,
83     },
84 ]
85
86 results = retrieve_memories(
87     query=query,
88     query_embedding=query_embedding,
89     memories=memories,
90     top_k=2,
91 )
92
93 print("Retrieved memories:")
94 for rank, item in enumerate(results, start=1):
95     print(
96         f"{rank}. {item.get('memory_id')} "
97         f"score={item.get('score'):.4f} "
98         f"semantic={item.get('semantic_score'):.4f} "
99         f"lexical={item.get('lexical_score'):.4f} "
100        f"importance={item.get('importance_score'):.4f}"
101    )
102
103 print("\nChecks:")
104 print_check("top-1 memory is m_space", len(results) > 0 and
105 results[0].get("memory_id") == "m_space")
106 print_check("top-2 memory is m_award", len(results) > 1 and
107 results[1].get("memory_id") == "m_award")
108
109 if __name__ == "__main__":
110     run_demo()

```

一些参数解释：

- `semantic_similarity` 建议使用 query embedding 与 memory embedding 的 cosine similarity;
- `lexical_overlap` 需要使用上方函数计算;
- `importance` 由大家在构建 Memory Pool 时赋值;

4.2 Task 1: 带有Memory的个体模拟系统

本任务要求构建一个 Memory-Augmented Role-Playing Agent。系统需要根据用户问题，检索回相关对话或场景记忆，并以该角色身份回答问题。任务形式是角色采访问答。实验过程中，至少需要比较以下配置：

1. 只输入问题；
2. Persona + 问题；
3. Persona + Retrieved Memories + 问题。

报告中需要回答：

1. Memory是否对最终表现有提升？
2. 如何保证召回的记忆对访谈回答是有价值的？
3. 你所构建的系统时延是多少？

4.3 Task 2: 构造训练数据并训练角色模型

本任务要求构造监督微调或偏好优化数据，训练一个不依赖外部 Memory 调用也能直接进行角色化回复的模型。

可以使用之前的课堂已经学过的所有训练方法，包括但不限于：

- LoRA/SFT；
- DPO / ORPO / SimPO；
-

以下是可以考虑的方案建议，但是不一定要完全遵循，可以自行思考其它方式：

- **方案 A：** 从角色对话数据中构造 instruction-response 数据。可以从 `dialogue/` 中抽取角色对话，把某一轮目标角色的发言作为 `output`，把 Persona、场景描述和前若干轮对话历史作为 `input`。
- **方案 B：** 如果选择 DPO / ORPO / SimPO，可以在已有 SFT 样本基础上构造 `chosen/rejected`。`chosen` 可以来自原始角色回答或经过清洗的高质量回答；`rejected` 可以用以下方式构造：
 - 通用助手式回答：回答正确但没有角色身份；
 - 错角色回答：使用另一个角色的事实或语气；
 - 事实冲突回答：故意替换关键职业、关系、经历或偏好；
 - 风格冲突回答：内容相关但语气明显不符合该角色。

报告中需要回答：

1. 你的训练方法是否能提升角色回答质量？
2. 尝试比较多种不同的训练方法，或者训练的组合所对应的最终效果。
3. 可以挑选N个角色，尝试单独在这个N个角色的数据各自训练一个模型。最终模型效果有变化吗？

4.4 Task 3：训练后模型与 Memory 系统结合

本任务要求将 Task 2 训练后的模型与 Task 1 的带 Memory 的智能体结合，构建一个更强的角色个体模拟系统。Task 1 中，Memory 使用的是基础模型；Task 2 中，训练模型不调用外部 Memory；Task 3 则要求比较：已经学过角色行为模式的模型，在额外看到检索记忆后，是否能够进一步提升角色一致性？需要比较的设置包括：

1. Base Model，只给出Persona
2. Base Model + Memory；
3. SFT Model，不使用 Memory；
4. SFT Model + Memory。

报告中需要回答：

1. 训练模型加 Memory 是否优于单独 SFT？
2. Memory 是否能弥补模型没有记住的角色事实？
3. Memory 是否会破坏已经学到的说话风格？
4. 你能训练模型，使其更会利用 Memory 吗？

4.5 Task 4：综合挑战任务

Task 4 是开放挑战任务，不指定具体方法。目标是在保持相同 Persona 的前提下，使系统同时满足：

1. 在角色个体模拟任务上表现较好；
2. 在通用指令数据集上性能能够维持或提升。

通用指令数据集的评估可以采用附件中提供的评估脚本，在验证集(dev.jsonl)上测试。但最终 leaderboard 上提交时请采用测试集(test.jsonl)运行结果。

你可以自由组合之前你学过的所有方法。为了维持模型通用能力，你可以从互联网寻找合适的通用指令微调数据集进行训练。**如使用外部公开数据，必须在报告中说明来源、规模和清洗方式；不得使用测试集或未授权闭源模型生成的数据。**

报告中需要回答：

1. 角色适配和通用指令跟随之间是否存在 trade-off？

2. 哪种方案在角色模拟和通用能力之间最均衡？
3. 如果继续改进，最值得优化的是检索器、Prompt，还是训练数据、训练算法？

5. 需要上传到实验平台的内容

5.1 最终版本的实验报告、脚本、结果合集

上传要求：

提交截止时间：6月7日23:59之前

- PDF 实验报告（不超过四页）（组长姓名_学号.pdf）
 - 内容结构自由组织，包含但不限于实验方案设计、结果对比、分析结论与关键思考；
- 主线任务过程脚本（`/task_scripts`）；
- 角色模拟预测结果（`/predictions`）；
- 主线任务中构造的数据样例（`/data_samples`）；
- 上传内容说明（`readme.md`）
 - 上传的脚本结构、各个脚本功能、任务实行情况说明。

注意：上传文件的大小不得超过200MB，不需要上传模型权重文件，只上传可复现的过程脚本与数据样例。

5.2 Leaderboard结果

上传要求：

提交截止时间：6月2日18:30之前

在实验平台上上传最终产生的测试集推理文件。

角色模拟数据输出格式：

代码块

```
1  {"id": "sample_0001", "character_id": "char_001", "prediction": "..."}

```

通用指令数据输出格式：

代码块

```
1  {"key": "sample_0001", "prediction": "..."}

```

注意角色模拟评估涉及 LLM-as-Judge，评估结果获取会有延迟，请提交之后间隔10分钟再查看结果。

最终leaderboard结果将按照角色模拟数据测试排名和通用指令数据测试排名做加权和计算，其中角色模拟部分占70%，通用指令部分占30%。

6. 补充说明

- 实验平台最多接受5次上传（报告及脚本集）和3次leaderboard提交。
- 请严格遵守模型使用限制，所有生成和训练必须基于指定模型。
- 任何作弊行为（如直接调用未授权的API、使用非指定模型生成答案）将导致零分处理。

祝实验顺利！